Airport Manager – Control the Chaos

CSE 4304 : Software Project Lab - I

Team 5

Name : Sheikh Alimun Alahi (ID : 230042115)

Name: Rakibur Rahman Ananta (ID : 230042128)

Name : Irfan Sadik Chowdhury (ID : 230042147)

The Big Picture: A Living Airport Simulation

- Headless CLI Application
- Dynamic State Management
- Real-time Event System
- Resource Allocation Logic

Why we chose Airport Simulation?

Airports are one of the best examples of **scheduling, resource allocation, and event handling**. Even a small airport needs solid planning to avoid conflicts.

1

Scheduling & Resource Allocation

Core constraint-based problem

2

Object-Oriented Programming

Encapsulated class design

3

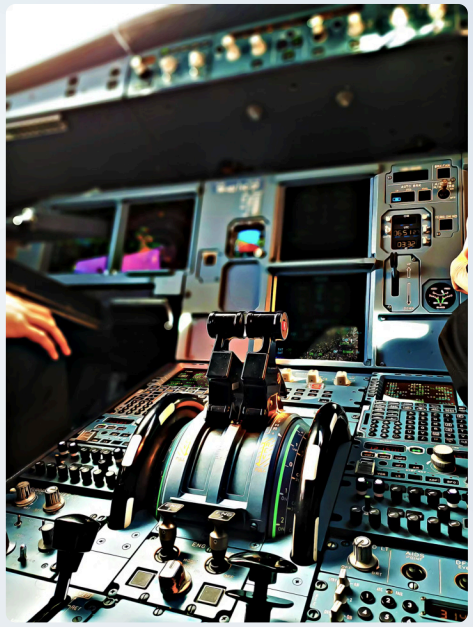
Data Structures & Modularity

Queues, lists, distributed logic

4

File-based Storage

Persistent state management



Key Features: A Real Airport Experience

Smart Gate Allocation

gate allocation + manual override

Dynamic Time Simulation

time-based simulation engine

Events & Challenges

event handling + random generation

Daily Performance Reports

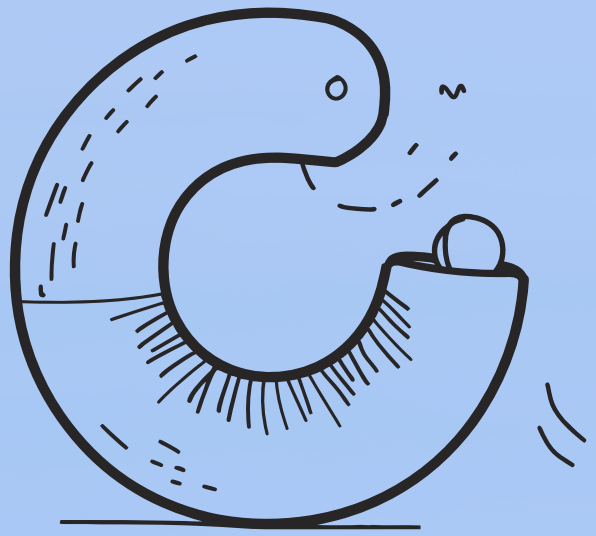
ASCII reports + analytics

Flight Management

Flight lifecycle tracking & state management

Passenger Check-in

Real-time passenger processing & boarding



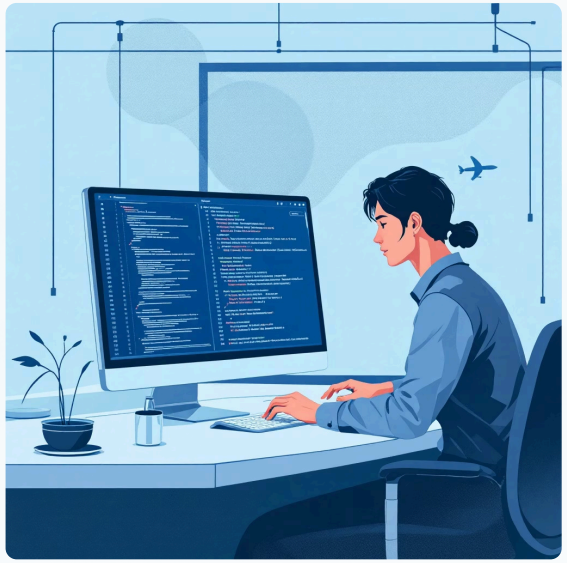
Tools Used

- C++
- Visual Studio Code
- Standard Library Only
- Github

Team Structure: Balanced Workload, Clean Architecture

Flights & Passengers Module

Flight lifecycle, passenger data, boarding logic

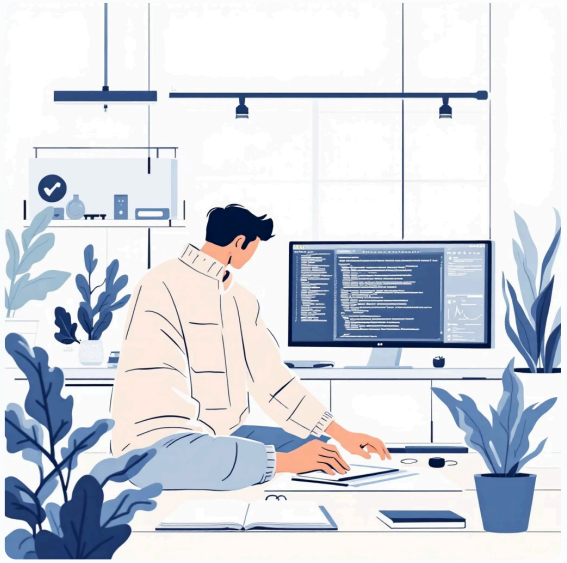


Gates & Simulation Engine

Gate allocation, time progression, state management

CLI, Events & Storage

Command interface, event handling, file persistence, reporting



Even distribution. Modular ownership. Unified system.

Development Timeline: Weekly Breakdown

Week 1–2 (December): Planning & Models

Class diagrams, Flight/Gate/ Passenger models, file structure

Week 3–4 (December): Core Managers

Flight Manager, Gate Manager, data linking

Week 5–6 (January): Passenger & Engine

Passenger Manager, time-tick system, user commands

Week 7–8 (January): Event System

Delays, emergencies, gate conflicts, random events

Week 9–10 (February): Storage & Reports

Save/load, ASCII reports, search/filter, Unit Testing & Integration

Week 11–12 (February): Testing & Polish

Integration testing, CLI refinement, performance checks

Week 13–14 (March): Final Sprint

Bug fixes, documentation, demo preparation

1 2 3 4 5 6 7

Ready for Takeoff?

Thank You!

Feel free to ask any Questions!